



Spring WebFlux

En mode Web Service REST Reactive

1

Spring WebFlux

Web Service REST Réactive

2

- ✓ Présentation
- ✓ Handler
- ✓ Route
- ✓ Asynchrone
- ✓ Client

Présentation

- Cette approche est disponible depuis le Spring v5 (~ fin 2018)
- Le Web Reactive ou WebFlux permet
 - ➡ d'être **en full asynchrone** côté back
 - ❑ Attention : les contrôleurs seront gérés de manière asynchrone par Spring, mais **c'est à vous** de gérer l'asynchrone côté @Service ou @Repository
 - ➡ de faire des web services Rest **en dissociant la route du traitement.**
 - ❑ Avec cette approche, vous pouvez brancher dynamiquement vos WS Rest

Présentation

- Vous pouvez aussi réaliser les traitements directement en lambda expression
 - Sans avoir à créer de classes en plus (mais au détriment de la lisibilité / maintenabilité)
- Pour marcher correctement **vous devrez** avoir un client **asynchrone**
 - Spring vous en fournit un, le **WebClient**
 - Pensez à vérifier que votre *front classique* est capable de recevoir des flux asynchrones

Mono & Flux

- Deux objets de type **Publisher** vont porter la 'magie' du reactive
- **Mono<T>** : retourne 0 ou 1 élément (ou une erreur)
 - Equivalent à Maybe de ReactiveX
- **Flux<T>** : retourne 0 ou N éléments (ou une erreur)
 - Equivalent à Flowable de ReactiveX
- Les deux sont des *Streams* **asynchrones** !!!!

Mono & Flux

- Pour fabriquer un objet Mono :

```
Mono<String> m = Mono.just("Hello man");
```

- Pour récupérer de manière **bloquante** la valeur du Mono

```
String v = Mono.block();
```

- Pour manipuler de manière **NON bloquante** la valeur du Mono

```
Mono.subscribe(result -> System.out.println(result));
```

Mono & Flux

➤ Pour fabriquer un objet Flux :

```
Flux<String> m = Flux.just("A", "B", "C");
```

```
List<String> iterable = Arrays.asList("A", "B", "C");  
Flux<String> seq2 = Flux.fromIterable(iterable);
```

Dépendances

- En Spring Boot, il faut ajouter la dépendance suivante :

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-webflux</artifactId>  
</dependency>
```

- N'oubliez pas d'y ajouter vos autres dépendances
 - ➡ Pour faire du Rest, du Spring Data ...
- **Attention** à votre serveur JEE embarqué, il doit pouvoir gérer le WebFlux

Approche par Handler

- Un Handler ressemble à un web service
- Chacune de ses méthodes va répondre à une Request
 - Prendra un objet `ServerRequest` en paramètre
 - `org.springframework.web.reactive.function.server.ServerRequest;`
- Chacune de ses méthodes va retourner un objet *Publisher* de type **Mono**<`ServerResponse`>
 - `org.springframework.web.reactive.function.server.ServerResponse`

Approche par Handler

- Le @Service ou le @Repository sera injecté dans le Handler
 - En @Autowired ou à la main
 - ❑ N'oubliez pas de déclarer votre Handler en @Component si vous y placez un @Autowired
- Attention : contrairement à un contrôleur classique, Il n'y a **aucune route** dans le Handler

Approche par Handler

11

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.MediaType;
import org.springframework.web.reactive.function.server.*;
import reactor.core.publisher.*;

@Component
public class PersonHandler {
    @Autowired
    private PersonRepository repository;

    public Mono<ServerResponse> listPerson(ServerRequest request) {
        Flux<Person> people = this.repository.findAll();
        return ServerResponse.ok().contentType(MediaType.APPLICATION_JSON).bodyValue(people);
    }

    public Mono<ServerResponse> createPerson(ServerRequest request) {
        Mono<Person> person = request.bodyToMono(Person.class);
        return ServerResponse.ok().bodyValue(this.repository.save(person.block()));
    }

    public Mono<ServerResponse> getPerson(ServerRequest request) {
        int personId = Integer.valueOf(request.pathVariable("id"));
        Mono<ServerResponse> notFound = ServerResponse.notFound().build();
        Mono<Person> personMono = this.repository.findById(personId);
        return personMono.flatMap(p -> ServerResponse.ok().contentType(MediaType.APPLICATION_JSON).bodyValue(p))
            .switchIfEmpty(notFound);
    }
}
```

Approche par Handler

- Notez que le repository (ou le service) retourne des **Mono<T>** ou **Flux<T>**
 - C'est à vous de faire en sorte que vos méthodes dans le `@Repository` 'classiques' retournent ce type de résultat
 - En Spring Data, il existe parfois ce type de repository (**ReactiveCrudRepository** en MongoDB par exemple)

```
@Repository
public class MonRepositoryPourWF {
    @Autowired
    private MonRepositoryNormal repo;

    public Mono<Person> findById(int id) {
        return Mono.just(repo.findById(id));
    }

    public Flux<Person> findAll() {
        return Flux.fromIterable(repo.findAll());
    }
    ...
}
```

Approche par Handler - Routes

- Vous pouvez créer votre/vos routes dans un @Bean
- Chaque route prendra en paramètre le/les handler nécessaires à son bon déroulement
 - C'est l'objet RouterFunction (et son helper RouterFunctions) qui porte la responsabilité de la route

```
@Configuration
public class GreetingRouter {

    @Bean
    public RouterFunction<ServerResponse> routeEnGet(PersonHandler handler) {

        return RouterFunctions
            .route(RequestPredicates.GET("/lister"))
            .and(RequestPredicates.accept(MediaType.APPLICATION_JSON)),
            handler::listPerson);
    }
}
```

Approche par @RestController

- Dans cette approche, on repart des WS classiques
 - On s'assure que les @Service/@Repository sont bien asynchrones
- Chaque méthode de WS va porter
 - La route (comme avant)
 - La commande HTTP (comme avant)
 - Les paramètres nécessaires (comme avant)
 - Va retourner un (pas comme avant)
 - ❑ Mono/Flux<T>
 - ❑ Mono/Flux<ResponseEntity<T>>

Approche par @RestController

```
@RestController
@RequestMapping("/comptes")
public class ListerCompteRestController {
    private static final Logger LOG = LogManager.getLogger();

    @Autowired
    private ICompteService compteService;

    @GetMapping("/lister")
    public Flux<CompteEntity> listerCompte(@RequestParam(value = "userId") ObjectId userId)
        throws FonctionnelleException, ErreurTechniqueException {
        // DAO qui retourne déjà un Flux<CompteEntity>
        return this.compteService.selectAll(userId);
    }
}
```

Gestion des erreurs

- La gestion des erreurs peut se faire comme pour nos WS Rest classiques
 - Avec `@ControllerAdvice` pour une gestion globale
 - Avec `@ExceptionHandler` pour une gestion plus locale
- N'oubliez pas
 - Vous retournez des Mono/Flux

Le client

- Spring vous permet de faire usage
 - d'un WebClient
 - ❑ Pour réaliser un client asynchrone de vos WS asynchrones
 - d'un WebClientClient
 - ❑ Pour se simplifier la vie dans les tests
- N'oubliez pas : vous allez recevoir des **Flux/Mono**, qui envoient l'information au fur et à mesure

WebClient

18

```
var client = WebClient.create(WebReactiveClient.ROOT_URL);  
Mono<UtilisateurEntity> resu = client.get().uri(uriBuilder -> uriBuilder  
    .path(WebReactiveClient.ROOT + "auth/{email}/{pwd}").build("df@aol.com", "df"))  
    .retrieve()  
    .bodyToMono(UtilisateurEntity.class);  
  
var d = resu.block(); // On attend la réponse (block = sync)  
LOG.info(d);
```

WebClientTest

```
@SpringBootTest
@AutoConfigureMockMvc
class AuthentifierRestControllerTest extends AbstractTest {

    @Autowired
    private WebTestClient client; // Pas besoin d'URL racine ici

    @Test
    void test() {
        this.client.get().uri("/auth/df@aol.com/df")
            .exchange().expectHeader()
                .contentType(MediaType.APPLICATION_JSON)
                .expectStatus().isOk() // Nos Assertions
                .expectBody()
                .jsonPath("$.nom").isNotEmpty()
                .jsonPath("$.nom").isEqualTo("Fargis");
    }
}
```

Repo en Full Async

- En Spring, vous pouvez réaliser vos couches `@Repository` en full asynchrone via
 - Le Spring Data
 - ❑ `ReactiveCrudRepository` (ou équivalent)
 - Cassandra, R2DBC, MongoDB, Neo4J, ElasticSearch, Redis
 - L'écriture manuelle de `@Async` sur vos méthodes
- Attention : dans le cas des bases de données relationnelles (R2DBC) la gestion des ordres en modifications (insert/update/delete) peut s'avérer être un réel casse-tête

Pour aller plus loin

➤ Article comparatif

- Spring MVC (API Rest classique)
- Spring WebFlux

<https://medium.com/deno-the-complete-reference/spring-boot-vs-spring-webflux-performance-comparison-for-hello-world-case-386da4e9c418#:~:text=At%20low%20concurrency%2C%20the%20median,order%20of%201%E2%80%93932ms%20only.>

Pour aller plus loin

22

➤ Quelques diagrammes, sur 5 millions de requêtes

